Widgets et mise en page

Séance 2 · état local et formulaires

Point de situation

Séance 1 : Flutter installé, Dart parcouru, le tutoriel officiel commencé.

Aujourd'hui commence **Pantry**, l'application construite jusqu'à la fin du semestre.

Aucun réseau, aucun état partagé, aucune navigation. Uniquement de la mise en page et de la saisie.

Aujourd'hui

Horaire	Bloc
0:00 – 0:15	Rappels
0:15 – 1:00	Widgets, cycle de vie, modèle de layout
1:00 – 2:00	TP 1 · Devine le rendu
2:00 – 2:15	Pause
2:15 – 2:55	`setState` et formulaires
2:55 – 3:55	TP 2 · fiche produit et formulaire
3:55 – 4:15	Débrief

Widgets

Cycle de vie et modèle de contraintes

Le `State` survit, le widget non

`StatelessWidget`, sa description ne dépend que de ses paramètres. Reconstruit quand ils changent.

`StatefulWidget`, associé à un objet `State` qui survit aux reconstructions. C'est le `State` qui porte les données mutables, pas le widget.

Le widget est jeté et recréé à chaque build. Le `State`, non. Cette séparation explique la totalité du cycle de vie qui suit.

Les quatre moments d'un `State`

Méthode	Quand	Équivalent Compose
`initState`	une fois, à l'insertion	`LaunchedEffect(Unit)`
`didUpdateWidget`	à chaque nouveau widget parent	recomposition avec nouveaux paramètres
`build`	à chaque reconstruction	le corps du `@Composable`
`dispose`	au retrait définitif	`DisposableEffect { onDispose { } }`

`mounted` indique si le `State` est encore dans l'arbre. À tester après tout `await` avant d'appeler `setState`.

Seul l'arbre des widgets est recréé à chaque build

Trois arbres parallèles, et c'est le premier qui trompe.

Arbre	Rôle	Durée de vie
Widget	configuration **immuable**	recréé à chaque build
Element	instance montée, fait le lien	persistant tant que le type ne change pas
RenderObject	mesure, positionne, peint	persistant, coûteux

Un widget n'est **pas** l'objet à l'écran. C'est une description, jetable, comparable à un `@Composable` réexécuté.

Reconstruire est bon marché

Allouer des widgets est une opération triviale.

Recalculer une mise en page et repeindre ne l'est pas, et Flutter ne le refait que là où la description a réellement changé.

Les contraintes descendent, les tailles remontent

`RenderPadding.performLayout`, à peine simplifié :

```
void performLayout() {
  final padding = _resolvedPadding;
  final inner = constraints.deflate(padding);          // ① contrainte réduite
  child!.layout(inner, parentUsesSize: true);          // ② l'enfant se dimensionne
  (child!.parentData! as BoxParentData).offset =
      Offset(padding.left, padding.top);               // ③ position
  size = constraints.constrain(Size(
    padding.horizontal + child!.size.width,
    padding.vertical + child!.size.height,
  ));
}
```

① les contraintes descendent · ② les tailles remontent · ③ le parent positionne.

`Padding` ne dessine rien : il réduit une contrainte et décale un enfant.

`const` supprime la reconstruction

```dart
const SizedBox(height: 16)      // objet unique, jamais reconstruit
SizedBox(height: 16)            // nouvelle instance à chaque build
```

Un widget `const` est un **objet unique en mémoire**. Flutter compare l'ancienne et la nouvelle description : elles sont identiques, la sous-arborescence est ignorée.

`very_good_analysis` le signale automatiquement. Le suivre.

Constraints go down.

Sizes go up.

Parent sets position.

Les contraintes descendent. Les tailles remontent. Le parent positionne.

Un enfant ne connaît jamais sa position

1. Un parent transmet à son enfant des **contraintes** : largeur et hauteur, minimales et maximales.
2. L'enfant choisit sa **taille** dans ces bornes, et la renvoie.
3. Le parent **place** l'enfant dans son propre repère.

Un enfant ne connaît jamais sa position. Un parent n'impose jamais directement une taille : il impose des bornes.

La quasi-totalité des surprises de mise en page vient d'un oubli d'une de ces trois étapes.

La page `docs.flutter.dev/ui/layout/constraints` déroule 29 exemples commentés à partir de cette seule règle. C'est la référence à ouvrir dès qu'une mise en page résiste.

Tight, loose, unbounded : la liberté laissée à l'enfant

```
SizedBox(width: 100, child: …)   // tight   : min = max = 100
Center(child: …)                 // loose   : min = 0, max = disponible
```

Contrainte	`min / max`	L'enfant…
tight	`min == max`	n'a aucun choix, sa demande est ignorée
loose	`min == 0`	choisit entre 0 et le maximum
unbounded	`max == infinity`	doit se dimensionner lui-même, sinon erreur

Ce sont les termes de l'API : `BoxConstraints.tight()`, `.loose()`, et `hasBoundedHeight`. Ils apparaissent tels quels dans les messages d'erreur.

Une contrainte `unbounded` transmise à un widget qui veut « tout l'espace » est une erreur, pas une approximation.

`Row` et `Column` laissent l'axe principal libre

Transmettent une contrainte **unbounded** sur leur axe principal : chaque enfant prend ce qu'il veut, la somme est calculée ensuite.

```
Row(children: [
  Expanded(child: A()),            // reçoit une contrainte tight
  Expanded(flex: 2, child: B()),   // deux fois plus large
  const Icon(Icons.star),          // se dimensionne lui-même
])
```

`Expanded` répartit l'espace **restant** au prorata des `flex`.

`Flexible` répartit de la même façon, mais laisse l'enfant prendre moins.

`Expanded` ne dessine rien

C'est une **annotation** posée sur un enfant, lue par le parent.

Seuls les `Flex` (`Row` et `Column`) savent la lire. Dans un `Stack`, l'annotation équivalente est `Positioned`.

```
Incorrect use of ParentDataWidget.
```

Ce message signale une annotation placée sous un parent qui ne la comprend pas. Il désigne le **parent immédiat**, pas un ancêtre lointain.

Rayures jaunes ou écran rouge

À l'écran	Nature	L'application
rayures jaunes et noires	dépassement de place	**continue de tourner**
écran rouge, exception	contrainte impossible	s'arrête

Un dépassement produit une mise en page fausse. Une contrainte impossible produit une absence de mise en page.

Les deux se lisent dans la console, et le message nomme le widget fautif.

Lire les contraintes avant d'ajouter un widget

```dart
import 'package:flutter/rendering.dart';

void main() {
  debugPaintSizeEnabled = true;   // bordures et espacements visibles
  runApp(const PantryApp());
}
```

Le **Widget Inspector** de l'extension Flutter affiche l'arbre et, pour chaque nœud, les contraintes reçues et la taille renvoyée.

Utiliser l'un des deux dès le premier symptôme, avant d'ajouter un widget au hasard.

Pas de `Modifier` : chaque effet est un widget

```
Modifier.padding(16.dp).background(Color.Red)      // Compose
```

```
Padding(                                            // Flutter
  padding: const EdgeInsets.all(16),
  child: ColoredBox(color: Colors.red, child: …),
)
```

Chaque effet est un widget qui en enveloppe un autre : l'arbre est plus profond, mais il n'existe qu'une notion, sans API parallèle.

Le réflexe demande une à deux séances. La page `docs.flutter.dev/flutter-for/compose-devs` traite précisément ce passage.

TP 1 · Devine le rendu

60 minutes

Six extraits. **Prédiction écrite avant exécution.**

`docs/seance-02-devine-le-rendu.md`

Pause

15 minutes

`setState`

État local et formulaires

`setState` reconstruit tout le `build()`

```
setState(() => _compteur++);
```

Deux choses, dans cet ordre :

1. exécuter la fonction passée, c'est là que l'état change ;
2. marquer le `State` comme à reconstruire.

La portée est le `build()` entier du `State`, pas la partie qui a changé. Un `setState` dans un widget haut placé reconstruit tout ce qui est en dessous.

D'où l'intérêt des widgets `const`, et des `State` proches de ce qu'ils modifient.

`setState` ne traverse pas les écrans

- Il ne reconstruit **pas** immédiatement : la reconstruction a lieu à la frame suivante.
- Il ne traverse **pas** les frontières : un autre `State` n'en sait rien.
- Il ne fonctionne **pas** après `dispose`. D'où le test de `mounted` après un `await`.

Le partage d'état entre écrans est le sujet de la séance 5.

Un formulaire en trois pièces

```dart
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: TextFormField(
    validator: (v) => (v?.isEmpty ?? true) ? 'Champ obligatoire' : null,
  ),
)

if (_formKey.currentState!.validate()) { … }
```

Pièce	Rôle
`Form`	regroupe les champs
`GlobalKey<FormState>`	donne accès à `validate()` et `reset()`
`validator`	renvoie `null` si valide, un message sinon

`validate()` ignore ce qui n'est pas un `FormField`

`validate()` interroge **les `FormField` descendants**, et rien d'autre.

Un sélecteur de date posé à côté du formulaire n'en est pas un : sa valeur ne sera jamais vérifiée, et aucune erreur ne le signalera.

Deux issues :

- contrôler la valeur à la main avant de soumettre ;
- envelopper le sélecteur dans un `FormField<DateTime>`, qui le fait entrer dans le cycle.

Un contrôleur oublié fuit en silence

```dart
final _controller = TextEditingController();

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}
```

Un contrôleur est un `Listenable` : sans libération, il reste référencé et fuit.

Aucune erreur n'est levée, aucun avertissement n'est émis, rien ne change à l'écran. C'est l'oubli le plus fréquent, et le plus silencieux.

Un `TextFormField` sans contrôleur fonctionne aussi : `validator` et `onSaved` suffisent souvent.

Le clavier réduit la hauteur disponible

Le clavier réduit la hauteur disponible. Un contenu de hauteur fixe se retrouve masqué.

Approche	Effet
`ListView` pour les champs	le contenu défile, le champ actif reste atteignable
`resizeToAvoidBottomInset`	contrôle si le `Scaffold` se redimensionne · `true` par défaut
`SingleChildScrollView`	équivalent pour un contenu non listé

Le choix du conteneur règle le problème. Ce n'est pas une option à activer.

Rendre un TP sur son fork

Le dépôt du cours est en lecture seule. Chacun le forke une fois, depuis `github.com/mduruisseau/pantry-2026`, bouton **Fork**.

```
git clone https://github.com/mduruisseau/pantry-2026.git
cd pantry-2026
git remote add perso https://github.com/<compte>/pantry-2026.git

git switch session-02-start
git switch -c tp02-<nom>
…
git add -A && git commit -m "TP 2"
git push perso tp02-<nom>
```

Les séances suivantes arrivent par `git fetch origin`. GitHub refuse le mot de passe du compte pour un push en HTTPS : un jeton d'accès personnel ou une clé SSH.

Quatre TP tirés au sort comptent pour 20 %

Règle	Détail
ce qui est vérifié	les critères de l'exercice prioritaire, section « Rendu » du `TP.md`
échéance	le dernier commit poussé avant la fin de la séance
binôme	chacun pousse sur son fork ; un rendu commun sur demande, `tp02-<nom1>-<nom2>`
tirage	quatre séances parmi les séances 2 à 10, après la séance 10
absence	une séance tirée et manquée vaut zéro, sauf absence justifiée

Pousser la branche de départ ou le corrigé ne valide rien.

TP 2 · Pantry

60 minutes en binôme

```
git switch session-02-start
git switch -c tp02-<nom>
flutter pub get
flutter run -d chrome
```

Énoncé complet dans `TP.md`, maquette dans `design/fiche-produit.svg`.

La fiche produit d'abord. Le formulaire peut être terminé hors séance.

Points de relecture

- Le rendu sur les **six** produits mockés, pas seulement le premier
- Le traitement des champs absents : ligne supprimée, texte de repli, valeur par défaut du modèle
- Le sélecteur de date : contrôle manuel, ou `FormField<DateTime>`
- Les ressources libérées dans `dispose`

Séance 3

Listes, navigation, thème et transitions.

Une question à garder de côté : dans une application à plusieurs écrans, comment un compteur placé dans l'AppBar connaîtrait-il le produit affiché ailleurs ?

`setState` n'y répond pas.

Questions

Pointeurs

Ressource	Adresse
Contraintes	`docs.flutter.dev/ui/layout/constraints` · 29 exemples
Flutter pour développeurs Compose	`docs.flutter.dev/flutter-for/compose-devs`
Catalogue de widgets	`docs.flutter.dev/ui/widgets`
Widget Inspector	`docs.flutter.dev/tools/devtools/inspector`
`Form` et validation	`api.flutter.dev/flutter/widgets/Form-class.html`
Cookbook · formulaires	`docs.flutter.dev/cookbook/forms`
Dépôt du cours	`github.com/mduruisseau/pantry-2026` → `session-02-start`